

URL Shortener Web Application(Advanced Version)

Internship Project – Innomatics Research Labs

1. Introduction

In the modern digital era, URLs are widely used to share websites, online resources, documents, and multimedia content. However, many URLs are lengthy and complex, which makes them difficult to copy, share, and remember. A URL Shortener application solves this problem by converting long URLs into shorter, easy-to-share links.

The aim of this project is to design and develop an Advanced URL Shortener Web Application using Flask. Unlike a basic URL shortener, this advanced version supports user authentication, allowing users to sign up, log in, and manage their own shortened URLs securely. Each user can view only their own URL history, making the application more personalized and secure.

This project was developed as part of my internship at Innomatics Research Labs, and it provided practical exposure to backend development, authentication, database design, and full-stack application workflow.

2. Objectives of the Project

The main objectives of this project are:

- To shorten long URLs into compact, shareable links.
- To allow users to sign up and log in securely.
- To associate shortened URLs with specific users.
- To store user and URL data using a database ORM.
- To redirect shortened URLs to their original URLs.
- To display user-specific URL history after login.
- To validate user inputs and URLs for reliability.

3. Technology Stack Used

The following technologies were used to develop the project:

Backend

Python – Core programming language.

Flask – Lightweight web framework for routing and request handling.

Flask-Login – Used to manage user authentication and sessions.

Frontend

HTML – Used to create the user interface pages such as signup, login, and dashboard.

Database

SQLite – Lightweight relational database for storing user and URL data.

SQLAlchemy ORM – Used to interact with the database using Python objects instead of raw SQL.

Validation validators library – Used to validate URLs before shortening and storing them

3. System Architecture

The Advanced URL Shortener Web Application follows a three-layer architecture:

1. Frontend Layer

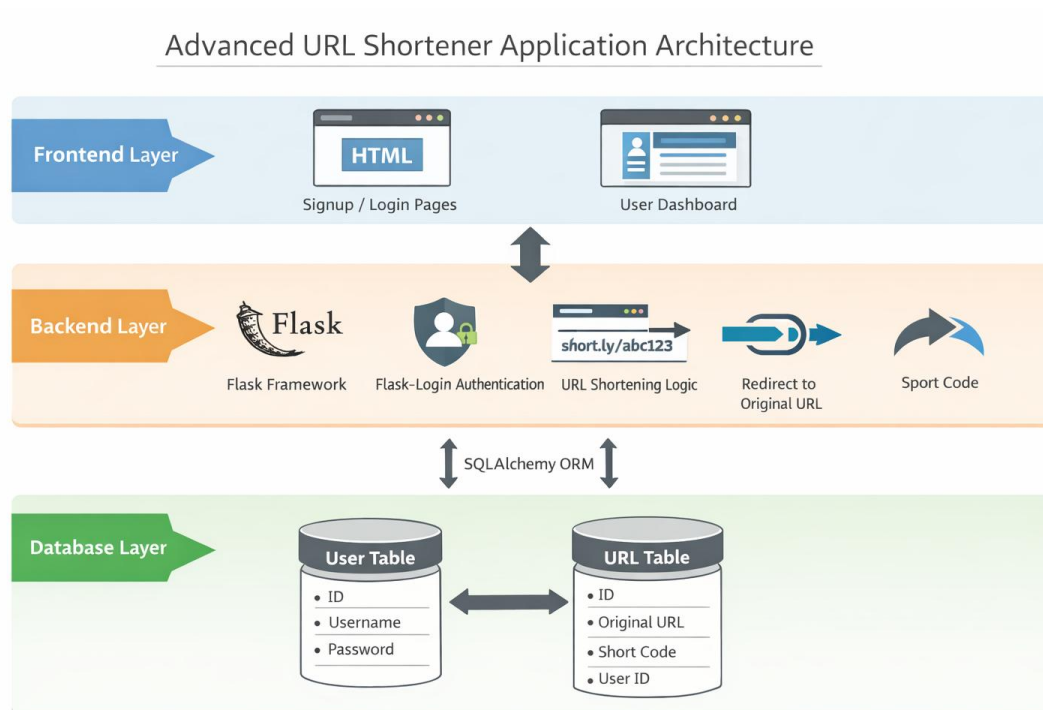
The frontend is developed using HTML and provides interfaces for user signup, login, and dashboard access. Users interact with the application through a web browser.

2. Backend Layer

The backend is built using Flask and handles routing, authentication, URL validation, short code generation, redirection logic, and session management using Flask-Login.

3. Database Layer

The database layer uses SQLite for persistent storage. SQLAlchemy ORM is used to manage database operations such as inserting users, saving URLs, and querying user-specific data.



4. Project Workflow

The workflow of the application is as follows:

- A new user visits the application and navigates to the signup page.
- The user creates an account by providing a valid username and password.
- Username validation ensures uniqueness and correct length.
- After successful signup, the user logs in using valid credentials.
- Once logged in, the user is redirected to the dashboard.
- The user enters a URL to shorten.
- The backend validates the URL.
- A unique short code is generated.
- The original URL and short code are stored in the database with the user's ID.
- The shortened URL is displayed on the dashboard.

- Clicking the short URL redirects the user to the original website.
- The user can log out securely from the application.

6. Database Design

The database consists of two main tables:

User Table

id – Unique identifier for each user.
username – Unique username (5–9 characters).
password – User password.

URL Table

id – Unique identifier for each URL.
original_url – The long URL entered by the user.
short_code – Generated short code.
user_id – Foreign key linking the URL to a specific user.

This design ensures that each user can access only their own shortened URLs

7. URL Validation

- The application uses the validators library to check the validity of URLs.
- Validation is performed before generating the short URL.
- Invalid URLs are rejected with an appropriate error message.
- This prevents incorrect data from being stored in the database.
- URL validation improves the reliability and robustness of the application.

8. Authentication and Authorization

User authentication is implemented using Flask-Login. The following features are included:

- Signup functionality with username validation.
- Login functionality with session management.
- Protected dashboard route using login_required.
- Logout functionality to end the user session.

This ensures that only authenticated users can access the dashboard and manage URLs.

8. Features Implemented

- User signup and login system.
- Username length and uniqueness validation.
- Secure session handling.
- URL shortening with random short code generation.

- URL redirection functionality.
- User-specific URL history.
- Logout functionality.
- Clean separation of frontend, backend, and database layers.

9. Challenges Faced

- During the development of this project, several challenges were encountered:
- Understanding Flask routing and dynamic URL handling.
- Implementing authentication using Flask-Login.
- Linking URLs to specific users in the database.
- Managing sessions and protected routes.
- Debugging template and import-related errors.

These challenges helped strengthen my debugging skills and understanding of Flask-based applications.

10. Conclusion

The Advanced URL Shortener Web Application was successfully developed using Flask, SQLAlchemy ORM, and SQLite. The application fulfills all the requirements specified in the project instructions, including authentication, URL shortening, redirection, and user-specific URL management.

This project provided valuable hands-on experience in backend development, authentication, database modeling, and real-world application design. It also improved my confidence in building scalable web applications using Flask.

11. Future Enhancements

The application can be further enhanced by:

- Hashing passwords for better security.
- Adding password reset functionality.
- Improving the user interface using CSS and Bootstrap.
- Adding analytics for URL clicks.
- Deploying the application to a cloud platform.